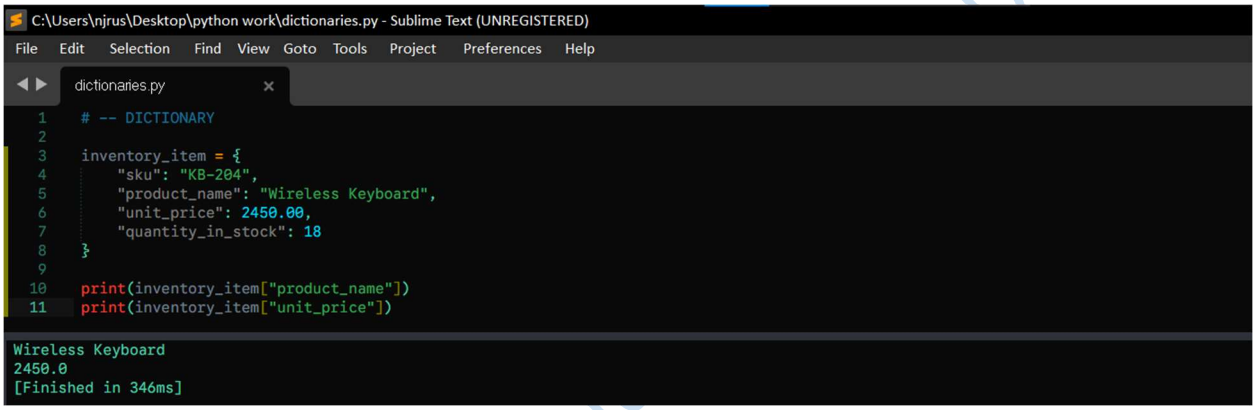
Working With Dictionaries

A dictionary becomes useful when a program can retrieve the information it needs and change that information when circumstances change.

Accessing Dictionary Values Using Keys

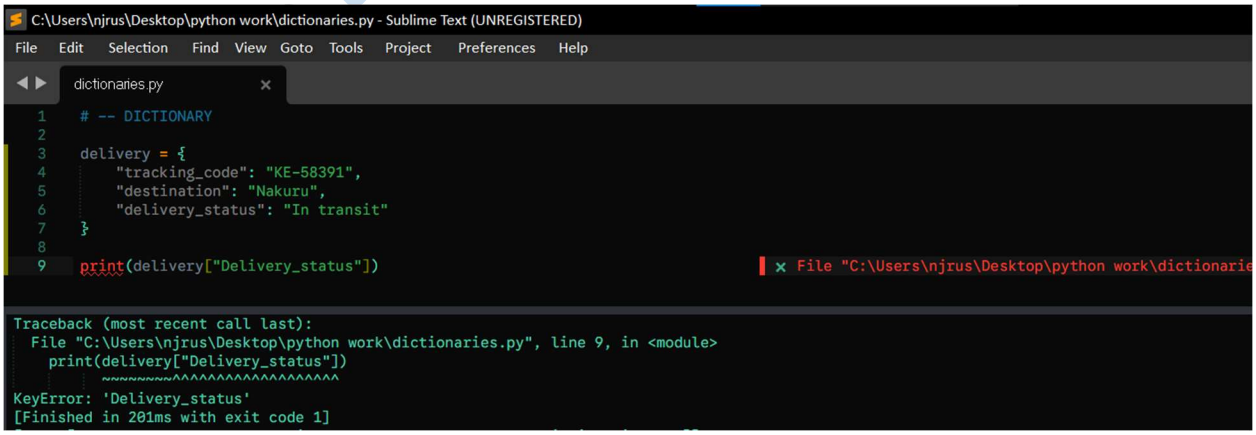
A dictionary stores information as key–value pairs. Unlike a list, a dictionary is not normally accessed using numeric positions such as `item[0]`. You access a dictionary entry by its **key**.

To retrieve a value, place the key inside square brackets after the dictionary's name. Python uses that key to find the associated value.



In `inventory_item["product_name"]`, the text `"product_name"` is the key. Python looks up that key and returns `"Wireless Keyboard"`. The same principle applies to the price.

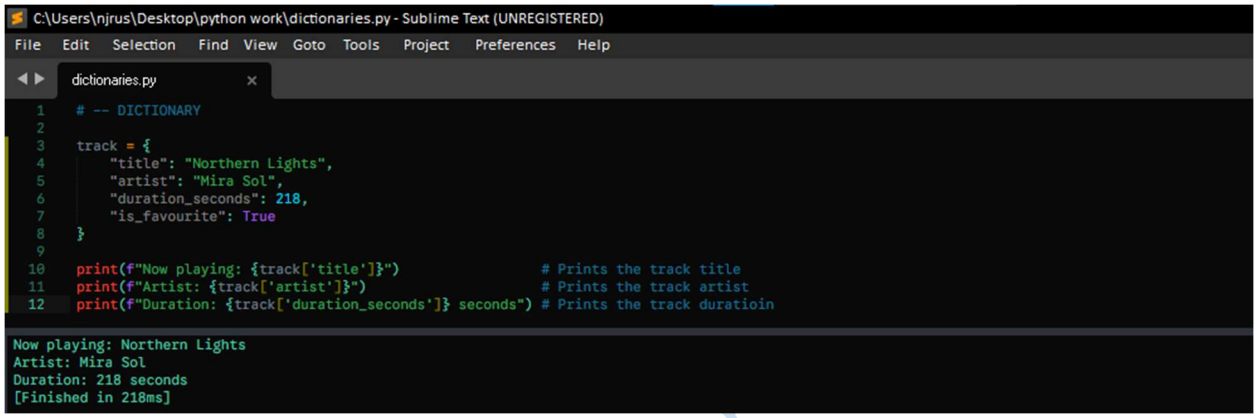
Note: The key you use to retrieve a value in a dictionary must match the key stored in the dictionary. For string keys, i.e., keys of type str, spelling, spaces, and capitalization matter. See below:



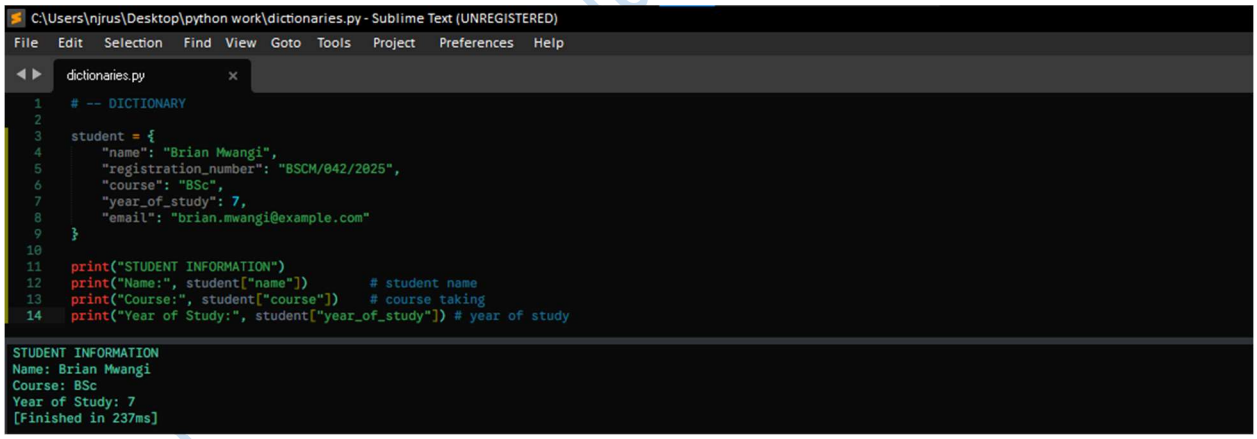
Here, we try checking the delivery status of our product but fails since the key does not exist. Trying to retrieve a value using keys that do not exist will raise a KeyError which means that python could not find the requested key.

More examples on accessing items

A music player program

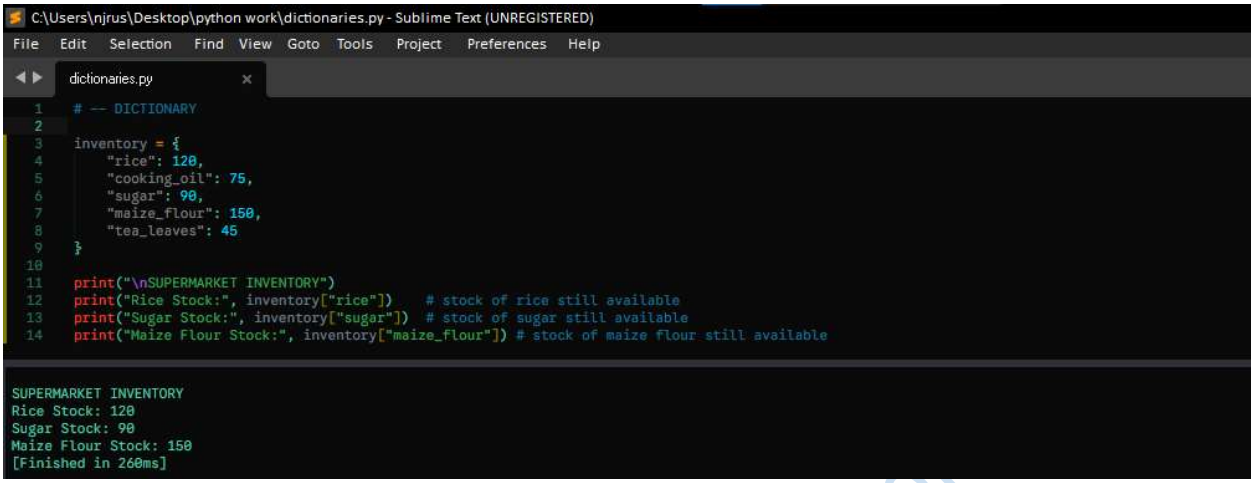


Student information

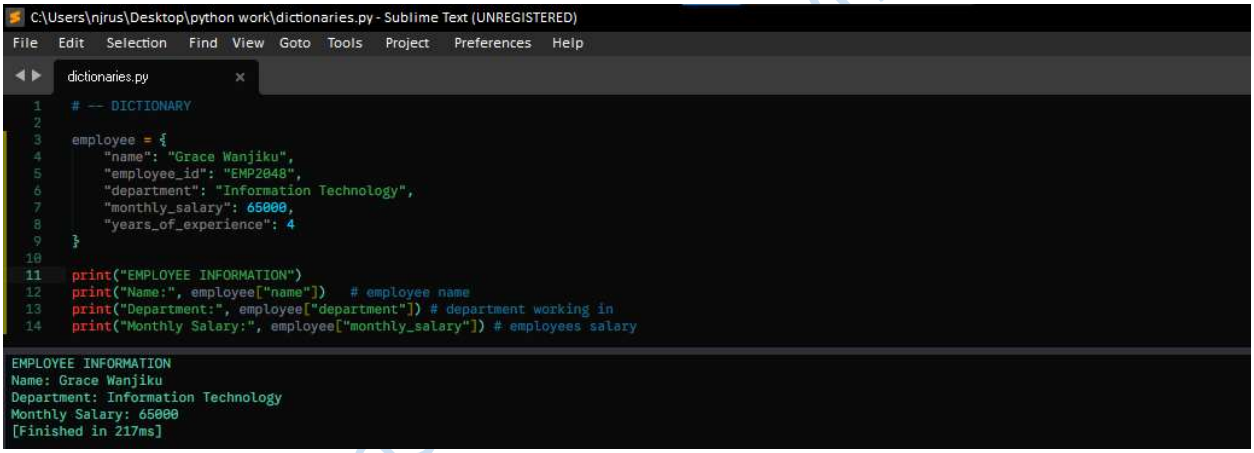


FROM BASICS TO JUNIOR PYTHON DEV: Data Structures: Dictionaries

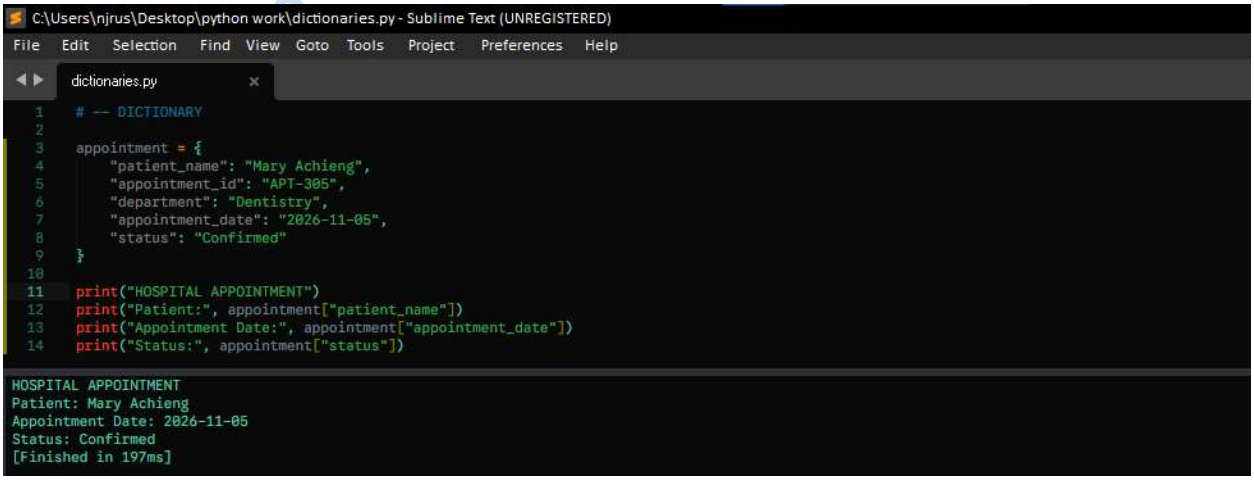
Supermarket inventory



Employee information



Hospital appointment

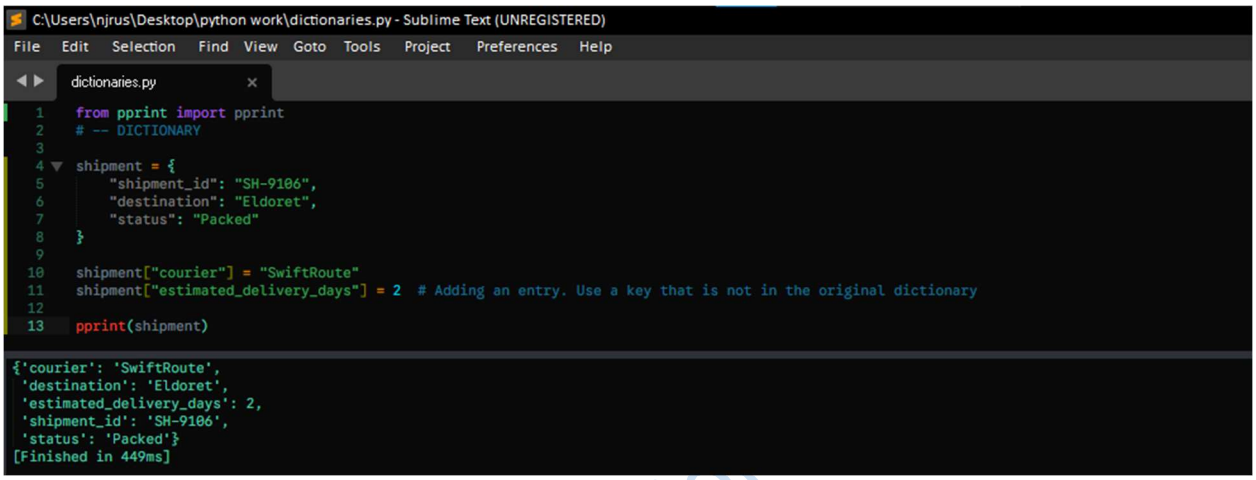


Adding and Updating Dictionary Entries

You can also add or update existing dictionary entries. To add an entry, **you need to use a key that is not in the dictionary**. To update an entry, **use the corresponding key available in the dictionary.**

To add a new entry, the general pattern is:

`dictionary_name[new_key] = value`



The dictionary 'shipment' now also contains the courier and the estimated delivery time in days. Because the keys "courier" and "estimated_delivery_days" were not already present, Python adds them with the values provided. The existing entries remain unchanged.

Now, to update an entry, use the same assignment syntax **but provide a key that already exists**. Python replaces the value associated with that key; it does not create a second entry with the same key.



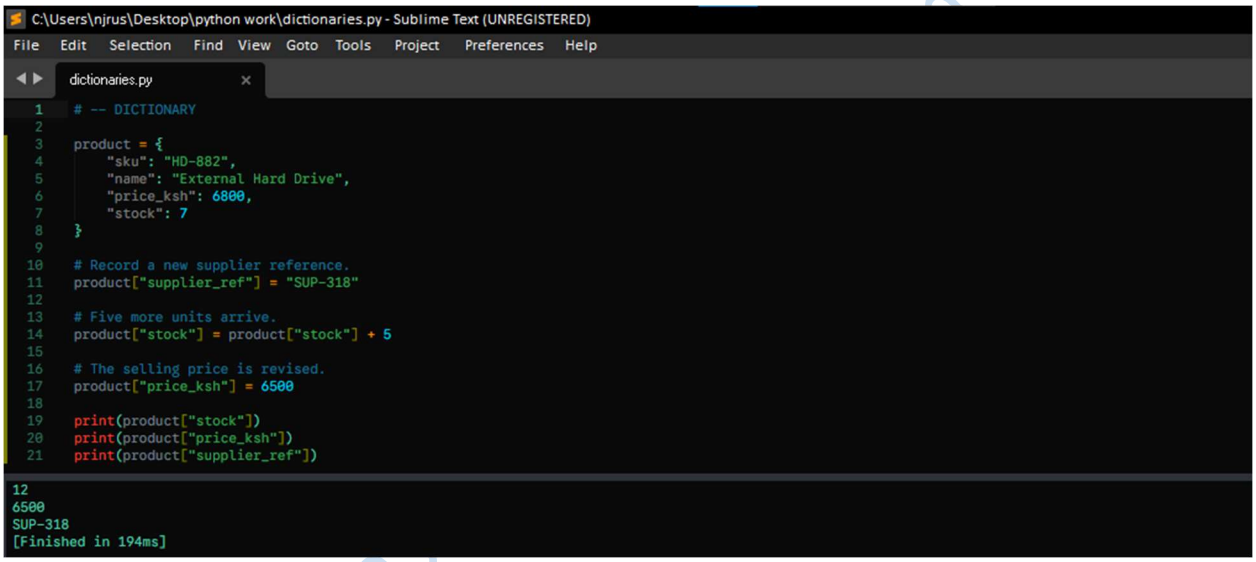
The status changed from "Packed" to "Dispatched". The key remains "status"; only its value changes.

The important distinction is whether the key already exists since the syntax for adding and updating does not change.

Situation	Code	Result
Key does not exist	profile["phone"] = "0712 345 678"	Adds a new entry
Key already exists	profile["account_status"] = "Active"	Replaces that key's value

Before writing the line, ask: is this information new, or am I changing information already stored?

An online shop may store a product's current stock and price. When a delivery arrives, the stock quantity increases. If the shop changes its price, the price value is updated.



The supplier reference is added because it is new. The stock and price are updated because those keys already existed. The stock expression reads the old value, adds five, and stores the new total.

Some common mistakes to avoid

- **Using a key that does not match**: Dictionary keys are case-sensitive. Using `record["Account_Status"]` when the actual key is `"account_status"` raises a KeyError.

- **Thinking assignment always adds a new entry:** Assigning a value using `record["status"] = "Complete"` updates the existing value if the key already exists; otherwise, it adds a new entry.

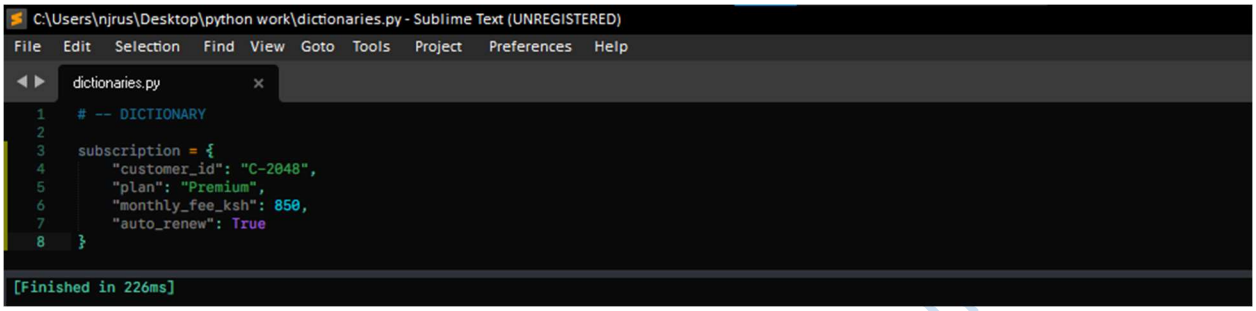
- **Using a list-style numeric index:** Writing `record[0]` looks for the key `0`. It does not access the first entry in the dictionary.

- **Accidentally overwriting information:** Assigning `results["Mathematics"] = 91` replaces the existing value associated with that key. Always confirm whether the replacement is intended.

Try Yourself

1. Retrieve account information

A service stores a customer's subscription details. Print the plan name and whether automatic renewal is enabled.

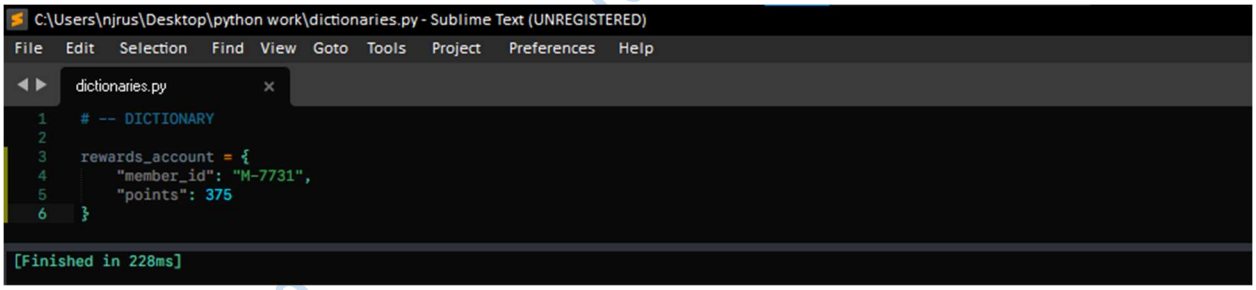


Add a "support_email" key with the value "help@example.com" to the subscription dictionary above.

Change "monthly_fee_ksh" to 950 and then print the updated fee.

2. Update using the existing value

A rewards account currently has 375 points. Add 80 points to the existing total and print the new balance.



3. Spot the problem

Given the dictionary below, explain why the first line works but the second line raises a KeyError.

